

Modelado de Datos, operaciones CRUD y Sincronización Realtime con Supabase

1. Introducción y Objetivos de la Sesión

Una vez resuelto el control de acceso de los usuarios, el núcleo de cualquier aplicación empresarial es la manipulación y consumo de información. Ahora pasaremos de la autenticación al diseño de entidades de datos estructuradas en la nube.

Trabajar con **Supabase** implica adoptar el paradigma **Relacional (SQL)**. A diferencia de las bases de datos NoSQL basadas en documentos (como Firebase), aquí cada registro debe respetar restricciones de integridad, tipos de datos estrictos y políticas de acceso. El objetivo principal es mapear tablas relacionales de PostgreSQL a objetos de programación en Dart, realizar operaciones CRUD eficientes y entender cómo suscribir la aplicación a un canal de comunicación asíncrono bidireccional (WebSockets) para actualizar la interfaz de usuario en tiempo real sin saturar el servidor mediante peticiones constantes (*polling*).

2. Fundamentos Técnicos

A. Mapeo Objeto-Relacional (ORM) Manual en Dart

En Flutter, para enviar datos a la nube o recibirlos, debemos cruzar una frontera de formatos. Supabase habla **JSON** (a través de su extensión PostgREST), mientras que nuestra interfaz de usuario habla **Objetos Dart**.

- **Deserialización (`fromMap`):** Transforma un mapa de datos `Map<String, dynamic>` proveniente de una fila de la base de datos SQL a una instancia de una clase Dart utilizable por los widgets.
- **Serialización (`toMap`):** Transforma las propiedades de un objeto Dart a un formato de mapa compatible con un payload JSON para su inserción o actualización remota.

B. El Protocolo Realtime mediante WebSockets

Tradicionalmente, para saber si hay datos nuevos en el servidor, una app móvil tendría que realizar peticiones HTTP **GET** cada ciertos segundos. Esto destruye la batería del dispositivo y consume ancho de banda innecesario.

Supabase resuelve esto mediante su servicio **Realtime**. Escucha el registro de replicación interno de PostgreSQL (*Write-Ahead Log* o WAL). Cuando ocurre un evento (**INSERT**, **UPDATE** o **DELETE**), el servidor de Supabase lo detecta y lo transmite de forma inmediata a todas las aplicaciones móviles conectadas a través de una única conexión persistente de **WebSockets**. En Flutter, este flujo continuo de datos se consume de forma nativa a través de un **Stream**.

3. Preparación en la Consola Web de Supabase

Para este laboratorio, crearemos una tabla de tareas pendientes (*To-Do List*) vinculada directamente al usuario autenticado.

Abran el **SQL Editor** en su consola de Supabase y ejecuten el siguiente script para crear la base de datos y activar los canales en tiempo real:

```
-- 1. Crear la tabla de Tareas
CREATE TABLE tareas (
  id BIGSERIAL PRIMARY KEY,
  user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE NOT NULL,
  titulo TEXT NOT NULL,
  completado BOOLEAN DEFAULT FALSE NOT NULL,
  creado_en TIMESTAMPTZ DEFAULT NOW() NOT NULL
);

-- 2. Habilitar la replicación en tiempo real para esta tabla específica
ALTER PUBLICATION supabase_realtime ADD TABLE tareas;
```

4. Laboratorio Código Fuente Paso a Paso (40 min)

Actualizaremos la arquitectura del proyecto para añadir el repositorio de datos. La estructura de archivos se expandirá de la siguiente manera:

```
lib/
├── domain/
│   ├── auth_repository.dart
│   └── tareas_repository.dart <-- NUEVO
├── infrastructure/
│   ├── supabase_auth_repository.dart
│   └── supabase_tareas_repository.dart <-- NUEVO
├── presentation/
│   ├── auth_router.dart
│   ├── login_screen.dart
│   └── home_screen.dart <-- MODIFICADO
└── main.dart
```

Paso 1: Crear el Modelo de Datos (Entidad)

Esta clase representa una fila dentro de nuestra tabla de PostgreSQL.

lib/domain/tarea_model.dart

```
class Tarea {
  final int? id;
  final String userId;
  final String titulo;
  final bool completado;
```

```
Tarea({
  this.id,
  required this.userId,
  required this.titulo,
  this.completado = false,
});

// Convertir una fila de la BD (Map) a un Objeto Dart
factory Tarea.fromMap(Map<String, dynamic> map) {
  return Tarea(
    id: map['id'] as int?,
    userId: map['user_id'] as String,
    titulo: map['titulo'] as String,
    completado: map['completado'] as bool,
  );
}

// Convertir un Objeto Dart a un mapa JSON para enviarlo a la BD
Map<String, dynamic> toMap() {
  return {
    if (id != null) 'id': id,
    'user_id': userId,
    'titulo': titulo,
    'completado': completado,
  };
}
}
```

Paso 2: Definir el Contrato del Repositorio de Tareas

lib/domain/tareas_repository.dart

```
import 'tarea_model.dart';

abstract class TareasRepository {
  /// Stream que emite la lista completa de tareas en tiempo real
  Stream<List<Tarea>> escucharTareas(String userId);

  /// Insertar una nueva tarea en PostgreSQL
  Future<void> crearTarea(Tarea tarea);

  /// Actualizar el estado de una tarea (completada/pendiente)
  Future<void> actualizarEstadoTarea(int id, bool completado);

  /// Eliminar un registro físico de la base de datos
  Future<void> eliminarTarea(int id);
}
```

Paso 3: Implementación del Repositorio con el SDK de Supabase

Utilizaremos la sintaxis fluida del ORM nativo de Supabase para comunicarnos con PostgreSQL.

lib/infrastructure/supabase_tareas_repository.dart

```
import 'package:supabase_flutter/supabase_flutter.dart';
import '../domain/tarea_model.dart';
import '../domain/tareas_repository.dart';

class SupabaseTareasRepository implements TareasRepository {
  final SupabaseClient _supabaseClient = Supabase.instance.client;

  @override
  Stream<List<Tarea>> escucharTareas(String userId) {
    // Retorna un flujo reactivo directo de la tabla 'tareas' filtrado por el
    // usuario actual
    return _supabaseClient
      .from('tareas')
      .stream(primaryKey: ['id'])
      .eq('user_id', userId)
      .order('creado_en', ascending: false)
      .map((listOfMaps) => listOfMaps.map((map) =>
Tarea.fromMap(map)).toList());
  }

  @override
  Future<void> crearTarea(Tarea tarea) async {
    try {
      await _supabaseClient.from('tareas').insert(tarea.toMap());
    } catch (e) {
      throw Exception('Error al insertar tarea: $e');
    }
  }

  @override
  Future<void> actualizarEstadoTarea(int id, bool completado) async {
    try {
      await _supabaseClient
        .from('tareas')
        .update({'completado': completado})
        .eq('id', id);
    } catch (e) {
      throw Exception('Error al actualizar tarea: $e');
    }
  }

  @override
  Future<void> eliminarTarea(int id) async {
    try {
      await _supabaseClient.from('tareas').delete().eq('id', id);
    } catch (e) {
      throw Exception('Error al eliminar tarea: $e');
    }
  }
}
```

```

    }
  }
}

```

Paso 4: Rediseño de la Interfaz de Usuario (Dashboard Reactivo)

Modificaremos la pantalla principal (`HomeScreen`) para que renderice un listado reactivo controlado por un widget `StreamBuilder`. Cada cambio en la base de datos actualizará esta vista de forma inmediata sin necesidad de recargar la página.

`lib/presentation/home_screen.dart`

```

import 'package:flutter/material';
import '../domain/auth_repository.dart';
import '../domain/tareas_repository.dart';
import '../domain/tarea_model.dart';
import '../infrastructure/supabase_tareas_repository.dart';

class HomeScreen extends StatefulWidget {
  final AuthRepository authRepository;
  final String userId;

  const HomeScreen({super.key, required this.authRepository, required
this.userId});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  // Inicializamos el repositorio de datos de forma local para la vista
  final TareasRepository _tareasRepository = SupabaseTareasRepository();
  final _todoController = TextEditingController();

  void _agregarTarea() async {
    if (_todoController.text.trim().isEmpty) return;

    final nuevaTarea = Tarea(
      userId: widget.userId,
      titulo: _todoController.text.trim(),
    );

    try {
      await _tareasRepository.crearTarea(nuevaTarea);
      _todoController.clear();
    } catch (e) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(e.toString()), backgroundColor: Colors.redAccent),
      );
    }
  }
}

```

```
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Gestor de Tareas Cloud'),
      actions: [
        IconButton(
          icon: const Icon(Icons.logout, color: Colors.redAccent),
          onPressed: () => widget.authRepository.signOut(),
        ),
      ],
    ),
    body: Padding(
      padding: const EdgeInsets.all(16.0),
      child: Column(
        children: [
          // Sección de Input para Nuevas Tareas
          Row(
            children: [
              Expanded(
                child: TextField(
                  controller: _todoController,
                  decoration: const InputDecoration(
                    labelText: 'Nueva tarea universitaria...',
                    border: OutlineInputBorder(),
                  ),
                ),
              const SizedBox(width: 8),
              IconButton.filled(
                style: IconButton.styleFrom(backgroundColor: Colors.emerald),
                onPressed: _agregarTarea,
                icon: const Icon(Icons.add),
              ),
            ],
          ),
          const SizedBox(height: 20),

          // Listado de Datos Reactivo en Tiempo Real
          Expanded(
            child: StreamBuilder<List<Tarea>>(
              stream: _tareasRepository.escucharTareas(widget.userId),
              builder: (context, snapshot) {
                if (snapshot.connectionState == ConnectionState.waiting) {
                  return const Center(child: CircularProgressIndicator(color:
Colors.emerald));
                }

                if (!snapshot.hasData || snapshot.data!.isEmpty) {
                  return const Center(
                    child: Text('No hay tareas pendientes en la nube.'),
                  );
                }
              },
            ),
          ),
        ],
      ),
    ),
  );
}
```

```

    }

    final tareas = snapshot.data!;

    return ListView.builder(
      itemCount: tareas.length,
      itemBuilder: (context, index) {
        final tarea = tareas[index];
        return Card(
          child: ListTile(
            leading: Checkbox(
              activeColor: Colors.emerald,
              value: tarea.completado,
              onChanged: (bool? valor) {
                if (tarea.id != null) {
                  _tareasRepository.actualizarEstadoTarea(tarea.id!,
valor ?? false);
                }
              },
            ),
            title: Text(
              tarea.titulo,
              style: TextStyle(
                decoration: tarea.completado ?
TextDecoration.lineThrough : null,
              ),
            ),
            trailing: IconButton(
              icon: const Icon(Icons.delete_outline, color:
Colors.redAccent),
              onPressed: () {
                if (tarea.id != null) {
                  _tareasRepository.eliminarTarea(tarea.id!);
                }
              },
            ),
          ),
        );
      },
    );
  },
),
],
),
),
);
}
}

```